ЛАБОРАТОРНАЯ РАБОТА № 2

Проектирование схемы БД и архитектуры REST API TaskFlow с помощью YandexGPT

Программное обеспечение: браузер; сервис Яндекса на базе YandexGPT; GigaChat (для проверки решения); dbdiagram.io или draw.io для ER-диаграммы; текстовый редактор.

Форма отчётности: ЛР2_Отчёт_Фамилия.pdf (скриншоты, SQL-код, ER-диаграмма, таблица эндпоинтов) + файл schema.sql.

Цель: спроектировать реляционную схему базы данных и архитектуру REST API проекта TaskFlow, научиться проверять и дорабатывать предложенные технические решения.

Рабочее место: где и как выполнять работу

Эта работа не связана с программированием и не требует запуска кода. Всё, что нужно – браузер и обычный текстовый файл для сохранения итогового SQL-скрипта.

1. Откройте браузер и войдите в сервис Яндекса с ИИ (https://ya.ru/ai). Отдельная установка программ для этого не нужна.
2. На рабочем столе или в папке для учёбы создайте папку taskflow, если она ещё не создана. В неё в дальнейшем будут добавляться файлы проекта из следующих работ.
3. Внутри taskflow создайте пустой текстовый файл с именем schema.sql – в него в конце работы нужно будет скопировать итоговый SQL-код.
4. Для ER-диаграммы отдельная программа не нужна: сайт dbdiagram.io (или draw.io) открывается прямо в браузере, регистрация не обязательна.

> **Если удобнее работать в VS Code:** можно уже сейчас установить редактор с сайта code.visualstudio.com и открыть в нём папку taskflow (Файл → Открыть папку). Это не обязательно для текущей работы, но пригодится начиная с ЛР № 5, где VS Code понадобится постоянно.

Файл schema.sql на этом этапе не запускается и не выполняется – база данных появится позже, когда в лабораторной работе № 8 схема будет подключена к Flask-приложению. Пока это просто текстовый файл с сохранённым кодом для дальнейшего использования.

Теоретическая справка

Таблицы, ключи и связи

В реляционной базе данных информация хранится в таблицах, а не в одном большом списке. У каждой таблицы есть набор столбцов (полей) с определённым типом данных и строки – конкретные записи. Чтобы однозначно отличать одну строку от другой, в таблицу добавляют первичный ключ (PRIMARY KEY) – поле, значение которого не повторяется ни у одной другой строки этой же таблицы. Обычно для этого используют числовой идентификатор id, который увеличивается автоматически при добавлении новой записи.

Связи между таблицами описываются через внешний ключ (FOREIGN KEY): в одной таблице хранится значение первичного ключа другой таблицы. Например, у каждой задачи в таблице tasks есть поле project_id, которое указывает, какому именно проекту из таблицы projects принадлежит эта задача. Такая связь называется «один ко многим»: один проект может включать множество задач, но у каждой задачи есть только один проект.

```
CREATE TABLE projects (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL
);

CREATE TABLE tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    project_id INTEGER,
    FOREIGN KEY (project_id) REFERENCES projects(id)
);
```

Отдельно стоит обратить внимание на правило удаления связанных записей. Если удалить проект, у которого уже есть задачи, база данных должна знать, что делать с этими задачами: удалить вместе с проектом (ON DELETE CASCADE), оставить, но обнулить ссылку (ON DELETE SET NULL), или вовсе запретить удаление, пока у проекта есть задачи. Выбор конкретного варианта – часть проектирования схемы, а не техническая мелочь.

Типы данных в SQLite

SQLite устроен проще многих других СУБД: вместо десятков типов данных здесь используется всего несколько базовых – TEXT (строка), INTEGER (целое число), REAL (дробное число), BLOB (двоичные данные) и NULL (отсутствие значения). Дата и время отдельного типа не имеют и обычно хранятся как TEXT в формате ISO (например, 2026-09-20 12:00:00) или как INTEGER – количество секунд от начала эпохи. Для учебного проекта TaskFlow этого набора вполне достаточно.

ER-диаграмма

ER-диаграмма (Entity-Relationship, «сущность – связь») – это схема, на которой видно все таблицы будущей базы данных и то, как они связаны между собой, ещё до того, как написана хотя бы одна строка SQL. В нотации Crow's Foot («воронья лапка»), которую используют dbdiagram.io и draw.io, каждая таблица изображается прямоугольником со списком полей, а линия между таблицами заканчивается символом, показывающим, сколько записей одной таблицы может соответствовать одной записи другой: одна черта означает «ровно одна», «воронья лапка» – «много».

REST API и HTTP-методы

REST (Representational State Transfer) – способ построения API, при котором клиент обращается к серверу по обычным HTTP-запросам, а каждый адрес (эндпоинт) отвечает за одну сущность. Смысл запроса определяется не текстом в адресе, а HTTP-методом:

Метод	Что делает	Пример
GET	получить данные, ничего не изменяя	GET /tasks
POST	создать новую запись	POST /tasks
PUT	полностью заменить запись	PUT /tasks/5
PATCH	изменить часть полей записи	PATCH /tasks/5
DELETE	удалить запись	DELETE /tasks/5

Эндпоинты принято именовать существительными во множественном числе (/tasks, а не /getTasks), а вложенность адреса отражает вложенность данных: /projects/{id}/tasks означает «задачи конкретного проекта».

Ход работы

Шаг 1. Спроектируйте схему базы данных

Откройте YandexGPT (https://ya.ru/ai). Отправьте промпт для проектирования схемы БД:

> Спроектируй реляционную базу данных для веб-приложения TaskFlow (менеджер задач). Таблицы: users (id, username, email, password_hash, created_at), projects (id, name, description, owner_id, created_at), tasks (id, title, description, status, priority, project_id, assignee_id, created_at, deadline), comments (id, task_id, author_id, text, created_at).
>
> Укажи связи, первичные и внешние ключи. Выведи SQL CREATE TABLE для каждой таблицы под SQLite.

Сохраните полученный SQL-код в отчёте (текстом или скриншотом).

Шаг 2. Проверьте и доработайте SQL-код

Не переносите ответ ИИ в файл сразу. Сначала пройдите по чек-листу и отметьте каждый пункт «да», «частично» или «нет»:

Проверка SQL-кода	**Да / частично / нет**	**Комментарий**
В каждой таблице есть PRIMARY KEY		
FOREIGN KEY указаны корректно (tasks → projects, tasks → users, comments → tasks, comments → users)		
Заданы правила ON DELETE (CASCADE / SET NULL) там, где это уместно		
Типы данных соответствуют SQLite (TEXT, INTEGER, REAL, DATETIME)		
Поля status и priority присутствуют в tasks		

Если ИИ пропустил поля status (open / in_progress / done) или priority (low / medium / high) в таблице tasks, добавьте их самостоятельно прямо в код и отметьте это в отчёте.

Шаг 3. Постройте ER-диаграмму

1. Откройте dbdiagram.io (https://dbdiagram.io) или draw.io (https://app.diagrams.net).

2. Постройте ER-диаграмму по итоговой схеме: сущности, атрибуты, связи с указанием мощности (1–1, 1–∞).
3. Экспортируйте диаграмму как PNG и вставьте в отчёт.

Шаг 4. Получите список REST API-эндпоинтов

Запросите у ИИ архитектуру API:

> Составь список REST API-эндпоинтов для TaskFlow. Для каждого укажи: HTTP-метод, URL, описание, тело запроса и ответ (JSON). Включи CRUD для tasks, projects, users, comments, а также эндпоинты регистрации и входа.

Оформите таблицу эндпоинтов в отчёте (не менее 8 строк):

Метод	URL	Описание	Тело запроса / ответ
POST	/auth/register		
POST	/auth/login		
GET	/tasks		
POST	/tasks		
GET	/tasks/{id}		
PUT	/tasks/{id}		
DELETE	/tasks/{id}		
GET	/projects/{id}/tasks		

Шаг 5. Скорректируйте архитектуру

Сравните схему и список эндпоинтов с техническим заданием из ЛР № 1. Если что-то не совпадает – добавьте недостающие поля, уточните названия эндпоинтов, приведите их к единому стилю именования (существительные во множественном числе).

Шаг 6. Сохраните финальный SQL-скрипт

Скопируйте итоговый SQL-код в файл schema.sql, созданный в начале работы. Он понадобится в лабораторной работе № 8 при подключении базы данных к Flask/FastAPI через SQLAlchemy.

Оформление отчёта

Сохраните отчёт под именем ЛР2_Отчёт_Фамилия.pdf. Рекомендуемая структура:

1. Титульная часть: Ф. И. О., группа, дата, тема.
2. Промпт и SQL-код, полученный от YandexGPT.
3. Заполненный чек-лист проверки SQL-кода.
4. ER-диаграмма (PNG) с подписью инструмента, в котором она построена.
5. Промпт и таблица REST API-эндпоинтов.
6. Описание внесённых правок (шаг 5).
7. Ответы на контрольные вопросы.

8. Итоговый вывод объёмом 5–7 предложений.

В выводе ответьте: какие ошибки допустил ИИ при проектировании схемы и эндпоинтов; почему нельзя использовать сгенерированный SQL без проверки; чем вложенный ресурс (/projects/{id}/tasks) удобнее отдельного запроса с фильтром.

Контрольные вопросы

1. Что такое первичный ключ? Чем он отличается от внешнего ключа?
2. Какие типы данных поддерживает SQLite? Чем они отличаются от типов PostgreSQL?
3. Что означает ON DELETE CASCADE? Приведите пример использования.
4. Что такое нормализация базы данных? Что нарушает 1НФ?
5. В чём разница между GET и POST в REST API?
6. Что такое JSON и почему он стандартен для REST API?
7. Чем ER-диаграмма полезна при проектировании базы данных?

Работа считается выполненной, если

1. получена и сохранена схема БД (SQL CREATE TABLE) для всех четырёх таблиц;
2. SQL-код проверен по чек-листу, найденные недочёты исправлены;
3. построена и приложена ER-диаграмма с указанием связей;
4. получен и оформлён в таблицу список REST API-эндпоинтов;
5. архитектура скорректирована и расхождения с ТЗ из ЛР1 объяснены;
6. даны ответы на контрольные вопросы и сформулирован самостоятельный вывод;
7. отчёт сохранён в PDF, файл schema.sql приложен отдельно.

Полезные ссылки

1. Сервис Яндекса с ИИ: https://ya.ru/ai
2. GigaChat: https://giga.chat
3. dbdiagram.io: https://dbdiagram.io
4. draw.io (diagrams.net): https://app.diagrams.net
5. Документация SQLite: https://www.sqlite.org/docs.html